Designing a Scheduler for a Day in the Life of a Minervan (Part I)

Minerva University

CS110: Problem-Solving with Data Structures and Algorithms

Prof. Subasic

November 12th, 2024

Setting up

Daily tasks table:

Task ID	Description	Duration (min)	Dependencies	Type	Fixed-time	Importance	Deadline
0	Wake up and get out of bed	15	None	regular	None	Critical	9.30 am
1	Go outside	20	Task 0	regular	None	Moderate	15 pm
2	Grab coffee	20	Task 0, 1	regular	None	Moderate	15:40 pm
3	Take CS110 class	90	Task 0	regular	10 am	Critical	11:30 am
4	After Class Break	30	Task 0, 3	regular	11:30 am	Low	12 pm
5	Ride YouBike	15	Task 0	rotation -city	None	Low	None
6	Grab lunch at a local restaurant	40	Task 0, 5	rotation -city	None	Moderate	13:30 pm
7	Get Boba	10	Task 0, 5, 6	rotation -city	None	Low	17 pm
8	Complete CS111 PCW	90	Task 0, 6	regular	None	Critical	23 pm

Rotation-City-Centered Tasks:

Ride YouBike

YouBike is a local Taiwanese bike-sharing system, which provides the chance to experience the city like a local and become familiar with its layout. It allows for convenient exploration, taking me through different neighborhoods and streets at a slower, more personal space than the bus or metro. As I bike, I often discover small temples, street art, or local restaurants and shops that I would have otherwise missed.

Grab lunch at a local restaurant
Taipei has plenty of small restaurants offering authentic Taiwanese meals at low prices. Eating at one of these restaurants is a chance to interact with locals, often having to overcome language barriers, and engaging in a cultural exchange through food, flavors, and dining customs.

Get Boba
Taiwan is the birthplace of boba — a globally beloved sensation. The Boba shops are often bustling with locals and they offer multiple variations, allowing me to try different tea flavors, boba styles, and combinations. This is most definitely an integral (and tasty) part of Taiwanese culture.

Preparing Algorithmic Strategies

Video Link:
https://www.loom.com/share/9962dc72caf94ce398661571c646e9eb?sid=bc53681b-c227-44fc-941f-ae667f46853e

Working on Python implementation

A. Refer to Appendix III for OOP implementation of MaxHeapq

    a. Refer to Appendix III a for the test cases

B. Refer to Appendix IV for the implementation of Task and TaskScheduler

Here is the output with the tasks delineated in the table above, setting the starting time to 8:30 am

Running the scheduler:

🕰️ t=8h30
started 'Wake up and get out of bed' for 15 mins, with priority 82.5004
✅ t=8h45, task completed!
🕰️ t=8h45
started 'Go outside' for 20 mins, with priority 60.0021
✅ t=9h05, task completed!
🕰️ t=9h05
started 'Grab coffee' for 20 mins, with priority 60.0023
✅ t=9h25, task completed!
🕰️ t=10h00
started 'Take CS110 class' for 90 mins, with priority 45.0025
✅ t=11h30, task completed!
🕰️ t=11h30
started 'After Class Break' for 30 mins, with priority 40.0049
✅ t=12h00, task completed!
🕰️ t=12h00
started 'Ride YouBike' for 15 mins, with priority -2.4935
✅ t=12h15, task completed!
🕰️ t=12h15
started 'Grab lunch at a local restaurant' for 40 mins, with priority 50.0061
✅ t=12h55, task completed!
🕰️ t=12h55
started 'Get Boba' for 10 mins, with priority 50.0024
✅ t=13h05, task completed!
🕰️ t=13h05
started 'Complete CS111 PCW' for 90 mins, with priority 45.0073
✅ t=14h35, task completed!

🏁 Completed all planned tasks in 6h05min, and a total utility value of 430.0345!

C. Examples:

a. The scheduler prioritizes tasks based on their priority value (refer to Appendix IV a)

```
#Example for pririotizing based on priority value
task_1 = Task(0, "Task 1 - High Priority", 10, [], importance="critical", task_deadline=600)  #high priority task
task_2 = Task(1, "Task 2 - Moderate Priority", 20, [], importance="moderate", task_deadline=900)  #moderate priority task
task_3 = Task(2, "Task 3 - Low Priority", 30, [], importance="low", task_deadline=1200)  #low priority task

scheduler = TaskScheduler([task_1, task_2, task_3])
scheduler.run_task_scheduler(starting_time=8 * 60) #starting at 8am
```

```
Running the scheduler:

🕗 t=8h00
        started 'Task 1 - High Priority' for 10 mins, with priority 85.0095
        ✅ t=8h10, task completed!
🕗 t=8h10
        started 'Task 2 - Moderate Priority' for 20 mins, with priority 60.0003
        ✅ t=8h30, task completed!
🕗 t=8h30
        started 'Task 3 - Low Priority' for 30 mins, with priority 40.0026
        ✅ t=9h00, task completed!

🏁 Completed all planned tasks in 1h00min, and a total utility value of 185.0124!
```

The priority value is calculated using the following formula:

Priority = importance_score + urgency_score - duration_penalty - dependency_penalty

By intentionally assigning attributes that maximize or minimize this value, we can check if the code behaves as expected for the 3 tasks, and is thus making decisions based on priority values.

The output confirms this, and by returning the numerical value of the priority we can check they are in descending order.

b. Example demonstrating how changing input order yields the same output (refer to Appendix IV a)

```python
#Example changing order of input
task_4 = Task(3, "Task 1 - High Priority", 10, [], importance="critical", task_deadline=600)
task_5 = Task(4, "Task 2 - Moderate Priority", 20, [], importance="moderate", task_deadline=900)
task_6 = Task(5, "Task 3 - Low Priority", 30, [], importance="low", task_deadline=1200)

#scheduler with the same tasks in different order
scheduler_reordered = TaskScheduler([task_6, task_4, task_5])
scheduler_reordered.run_task_scheduler(starting_time=8 * 60)

#assert statements to confirm that the output order remains the same
assert scheduler.tasks[0].description == scheduler_reordered.tasks[1].description, "First task should be the high priority task"
assert scheduler.tasks[1].description == scheduler_reordered.tasks[2].description, "Second task should be the moderate priority task"
assert scheduler.tasks[2].description == scheduler_reordered.tasks[0].description, "Third task should be the low priority task"
```

```
Running the scheduler:

🕗 t=8h00
        started 'Task 1 - High Priority' for 10 mins, with priority 85.0096
        ✅ t=8h10, task completed!
🕗 t=8h10
        started 'Task 2 - Moderate Priority' for 20 mins, with priority 60.006
        ✅ t=8h30, task completed!
🕗 t=8h30
        started 'Task 3 - Low Priority' for 30 mins, with priority 40.0078
        ✅ t=9h00, task completed!

🏁 Completed all planned tasks in 1h00min, and a total utility value of 185.0234!
```

In this example the same three tasks are defined with the same attributes as in the first example, but they are passed to the scheduler in a different order. Despite the different input order, the scheduler still selects the tasks based on their computed priority values, and the order of execution is the same as in the first example. The assert statements confirm this.

D. Feasibility:

The schedule produced by my code appears generally feasible. It starts at 8:30 AM and ends at 2:35 PM, with activities like waking up, going outside, grabbing coffee, and attending class occurring at reasonable times during the day. This timing aligns well with a typical morning and early afternoon routine, fitting neatly within the usual structure of daily activities like classes and breaks.

However, the daily life of a Minervan is often unpredictable. We might stop to chat with people in the hallway or make spontaneous plans, which the scheduler cannot account for. This makes the rigid nature of the schedule challenging to test in practice—everything has specific time limits, with no allowance for spontaneous interactions or even procrastination.

Additionally, the scheduler assumes I'll wake up exactly at 8:30 AM. The deadline of 9:30 AM included as an attribute only affects the priority value calculation, it doesn't provide real flexibility. If I wake up feeling tired, I might let myself sleep another hour, rearranging the rest of my day—like getting coffee after class instead of before. The current design doesn't easily adapt to such real-life adjustments, which could make the schedule less practical.

However, it’s relevant to note that fixed-time tasks, such as CS110 class and the break after it, were handled correctly by the algorithm.

Test-drive Scheduler

Image 1. Grabbing coffee at Seven Eleven in the morning. I was a little early and already had my coffee by 09:05. I also had a 10 AM CS110 class planned that day, but since this was test-driven on a different day (a Tuesday), I used this slot to work on a CS110 assignment instead.

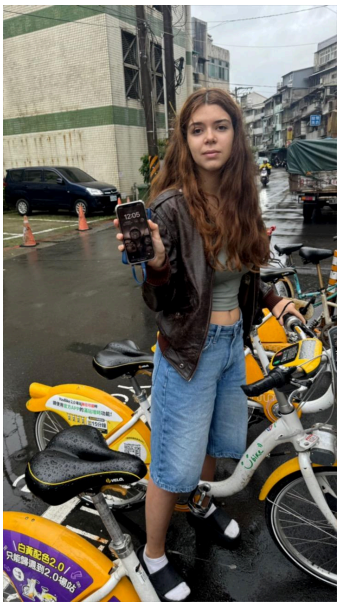

Image 2. Taking a YouBike to grab lunch at a local restaurant. It's 12:05, which is 5 minutes late to start biking.

Image 3. At 12:20 PM, I arrived at the local restaurant and placed my lunch order. This was also 5 minutes behind schedule, which started adding up.

Image 4. After sitting and eating at the restaurant I treated myself to some boba from the store next door. The time is 13:01, which is 6 minutes late. The next task was doing PCW at 13:05, which required me to be back at the reshall in 4 minutes (which did not happen).

Critical Analysis of the Algorithm

Advantages of Following the Output

The scheduler integrates task priorities well by considering aspects like urgency, importance, and dependencies. During the test, tasks such as waking up, grabbing coffee, and getting ready in the morning were prioritized to ensure that I was set up for a productive day.

By adhering to a strict timeline, the scheduler aims to maximize productivity, enabling me to accomplish several tasks in a compressed time frame. Tasks are still sequenced logically, due to the dependencies, so that I can smoothly transition between activities, like getting lunch after biking (since in real life, I biked to the restaurant).

Failure Modes and Patches

One of the critical issues with the scheduler is its rigidity. It assumes everything will go according to plan, without considering the real-world variability. For example, during my test drive, being 5 minutes late in one task resulted in a cascading delay for the rest of the day. The scheduler did not adapt dynamically to these delays, which would have been more helpful. A potential patch could involve adding more buffer times, but this could negatively impact the optimization of my productivity.

Although this was not the case, with the current algorithm it would be possible for a task to happen after its deadline, if there were multiple other tasks with a higher priority value. Currently, the deadline feature only influences the task's priority calculation, but since the formula is based on 3 other aspects, it is possible for tasks with longer deadlines to have higher priorities. For instance, after I introduced a new random task "Swimming", with a duration of 90 minutes and a deadline of 17 pm, this task took precedence over biking. Since grabbing lunch depends on biking, this was pushed back as well, and I ended up having lunch at 13:45 (which is 15 minutes after the designated deadline). This can be seen in the output below.

> Running the scheduler:
>
> 🕰️ t=8h00
>
> started 'Wake up and get out of bed' for 15 mins, with priority 82.5091
>
> ✅ t=8h15, task completed!
>
> 🕰️ t=8h15
>
> started 'Go outside' for 20 mins, with priority 60.0018
>
> ✅ t=8h35, task completed!
>
> 🕰️ t=8h35
>
> started 'Grab coffee' for 20 mins, with priority 60.0091
>
> ✅ t=8h55, task completed!

🕰️ t=10h00
started 'Take CS110 class' for 90 mins, with priority 45.0056
✅ t=11h30, task completed!
🕰️ t=11h30
started 'After Class Break' for 30 mins, with priority 40.0082
✅ t=12h00, task completed!
🕰️ t=12h00
started 'Swimming' for 90 mins, with priority 25.0091
✅ t=13h30, task completed!
🕰️ t=13h30
started 'Ride YouBike' for 15 mins, with priority -2.4982
✅ t=13h45, task completed!
🕰️ t=13h45
started 'Grab lunch at a local restaurant' for 40 mins, with priority 50.0033
✅ t=14h25, task completed!
🕰️ t=14h25
started 'Get Boba' for 10 mins, with priority 50.0053
✅ t=14h35, task completed!
🕰️ t=14h35
started 'Complete CS111 PCW' for 90 mins, with priority 45.0095
✅ t=16h05, task completed!

🏁 Completed all planned tasks in 8h05min, and a total utility value of 455.0628!

I could introduce an extra condition in the run_task_scheduler method, that checks whether the deadline of any task is approaching, and if so, prioritize that task even if others have higher priority values. This would ensure the times at which the tasks happen are logical, and I don't end up eating lunch at 18 pm.

General Efficiency of the Schedule

One of the metrics used was the completion rate of scheduled tasks. During the test drive, I managed to complete all the tasks listed in the schedule within the allotted time (although I had to rush). This was probably because I decided to overestimate the duration of each task. For instance, I completed CS111 PCW in 1 hour, which compensated for the fact that I was late 10min to start it. Either way, this 100% task completion rate indicates that, in theory, the scheduler successfully organized my day.

I also decided to evaluate the scheduler in how well it matched the tasks to the energy required to complete them. The goal would be to schedule high-energy tasks when I have high energy and reserve low-energy tasks for times when my energy is lower. This alignment would help maintain productivity without feeling overly fatigued. I rated the energy on a scale of 1-3 (3 being the most energy), and during the day I assessed my "Current Energy" level to see if it matched the activity. If my current energy matched it suggested the scheduler planned the task at a suitable time. This can be seen in the table below:

Description	Energy Required	Current Energy
Wake up and get out of bed	2	2
Go outside	3	2
Grab coffee	2	2
Take CS110 class	3	3
After Class Break	1	1
Ride YouBike	3	2
Grab lunch at a local restaurant	2	2
Get Boba	2	2
Complete CS111 PCW	3	1

As observed, later tasks such as "Complete CS111 PCW", which was scheduled at 13:05 after lunch, showed a mismatch with my energy level of 1. This mismatch suggests that this demanding task was scheduled during a low-energy period, making it challenging to focus and complete efficiently.

Overall, the scheduler achieved a 66% success rate in aligning tasks with my natural energy fluctuations. This indicates room for improvement, particularly in accounting for predictable dips in energy, such as the post-lunch dip in energy, to enhance productivity and reduce fatigue.

Experiments to Evaluate Scheduler Efficiency

Efficiency derived theoretically:

1. Main while loop: The loop iterates O(n) times, until all tasks are scheduled.
2. Calls get_tasks_ready O(n log n)
    a. Iterates over all tasks to check if they're ready to be scheduled O(n)
    b. Calls heappush() on the priority queue O(log n)
3. Finding the next fixed-time task: loop iterates over all tasks O(n)
4. Heappop(): removes the task with the highest priority from the heap O(log n)
5. Executing task: O(n log n)
    a. remove_dependency: iterates over all tasks to remove the completed task's ID from dependencies O(n)

b. after removing the dependency, get_tasks_ready is called again, which has a complexity of O(n log n) as explained above

So, overall: it can go up to O(n^2 log n)

Efficiency derived experimentally:

Refer to Appendix IV

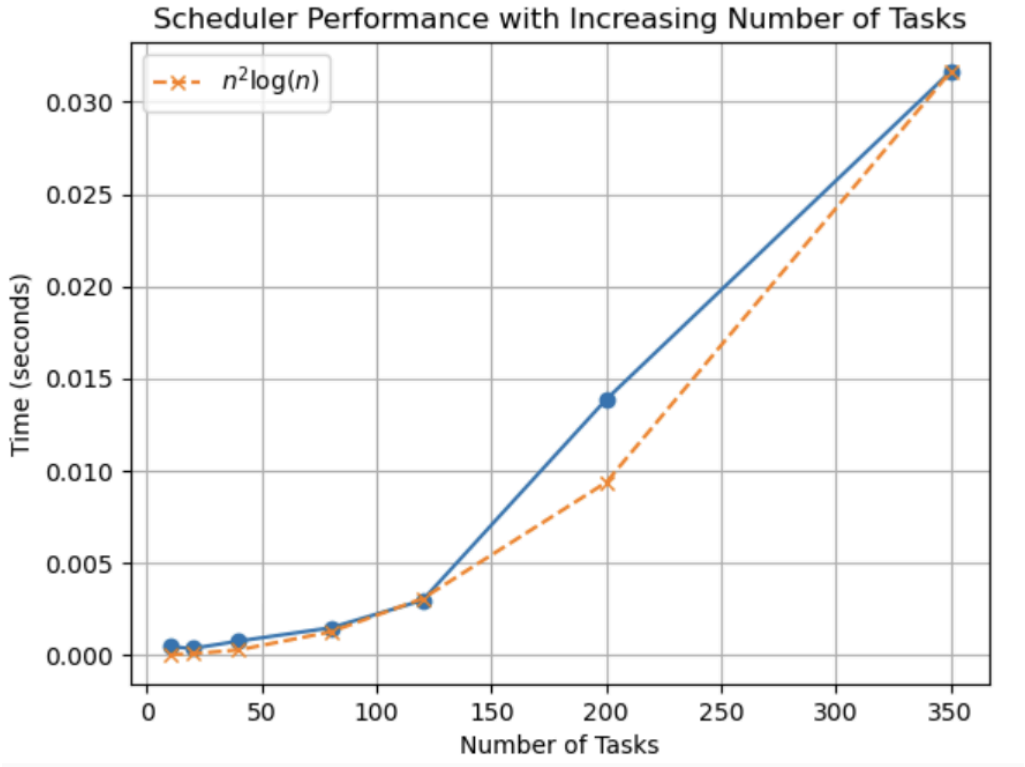

Figure 1. The graph shows the runtime of the task scheduler for different numbers of tasks, ranging from 10 to 350. The x-axis represents the number of tasks, while the y-axis shows the runtime in seconds. The data points were collected by running the task scheduler on randomly generated tasks, with increasing task sizes. Each point represents the time taken to schedule a given number of tasks, starting from 8:00 AM. The trend illustrates a gradual increase in runtime as the task count grows, aligning closely with the theoretical n2log(n) complexity, shown by the orange dashed curve.

Getting Back to the Board

Switching to more efficient data structures could help improve the scheduler. So in order to optimize it, I could maybe incorporate a Red-Black Tree for task prioritization. Unlike the current max-heap that uses a single priority value, a Red-Black Tree allows for prioritization based on multiple criteria, such as deadlines. Each task would be inserted into the tree using a composite key, such as (task_deadline, priority_value), ensuring that tasks are not prioritized solely based on the priority value. This prevents cases in which tasks might miss their deadlines due to less effective sorting (as exemplified before).

The self-balancing nature of Red-Black Trees ensures O(log n) complexity for insertion, deletion, and lookups. This means tasks with approaching deadlines can be dynamically reprioritized without a significant increase in complexity. For instance, as a task's deadline approaches, it remains appropriately positioned in the tree, ready for efficient retrieval.

Another enhancement could be using a dependency graph implemented via an adjacency list, which would streamline task readiness checks. In this setup, tasks are nodes, and dependencies are directed edges. In this sense, the algorithm could determine the order in which tasks should be executed, ensuring that tasks are only scheduled once their dependencies are resolved. This approach avoids repeatedly scanning through all tasks to verify dependencies, reducing time complexity and improving efficiency.

Word Count: 219

Appendix I: Reflection

The class discussion on heaps, priority queues, and scheduling strategies (session 13) was very helpful in guiding my problem-solving approach for this assignment. Understanding how max-heaps efficiently prioritize tasks by storing the highest priority at the root allowed me to use this structure for quick access to the most urgent tasks. Additionally, I chose to use a dictionary to store task IDs associated with priority values, rather than a list, since the lookup would be more expensive.

Word Count: 77

Appendix II: LOs and HCs

#AlgoStratDataStruct I selected a priority queue (implemented as a max-heap) and considered it's efficiency for task prioritization, being ideal for retrieving the highest-priority task in O(log n) time. I explained why this structure is appropriate by comparing it to alternatives like searching a list.

Word Count: 43

#ComplexityAnalysis I analyzed the scaling behavior of the scheduler. Based on previous feedback, I considered the different algorithm steps and how each contributes to the complexity. In order to check if theoretical results are corroborated with the experiments, I also included the theoretical curve in the graph (adjusting the scales), for easy comparison, confirming that scaling aligns with theoretical predictions.

Word count: 59

#ComputationalCritique I compared the priority queue with alternatives like sorted lists and Red-Black Trees. I explored test cases and explained potential scenarios in which the algorithm wouldn't be ideal (such as adding the "Swimming" task). I used different metrics such as runtime, task completion rate, and energy alignment rate, to evaluate performance demonstrating its strengths and limitations.

Word Count: 56

#PythonProgramming The code is fully functional, with the only intentional error being the expected one when attempting to heappop() from an empty heap. Test cases and examples verify priority-based scheduling and fixed-time task handling. I spend considerable time debugging, relying on strategies such as including print statements and even adding noise to priority values calculations as a response to errors.

Word Count: 59

#CodeReadability I ensured adherence to conventions, including meaningful function and variable names and consistent formatting. Comprehensive docstrings describe the purpose and usage of each function and comments throughout the code explain critical logic.

Word Count: 32

#Professionalism I delivered clear, concise communication in the written assignment. I ensured my approach was respectful of assignment guidelines and audience specifications, tailoring explanations for both technical and non-technical readers.

Word Count: 30

#organization I considered my audience and ensured clarity by dividing the report into topical sections that clearly corresponded to assignment instructions, facilitating grading. When necessary I also included snippets of code, tables, and calculations within the main text to facilitate lookups. When this wasn't necessary, I included links to appendix portions for easy reference. This was done in a consistent and logical manner, accumulating to a well-organized final result.

Word Count: 68

Appendix III: MaxHeap

The following implementation was adapted from Sousa, M. (2024, October 23). CS110 Session 13 - [7.2] Heaps and priority queues

```
class MaxHeapq:
    """
    A class that implements properties and methods
    that support a max priority queue data structure.

    Attributes
    ----------
    heap : arr
        A Python list where key values in the max heap are stored.
    heap_size: int
        An integer counter of the number of keys present in the max heap.
    """

    def __init__(self):
        """
        Initializes an empty heap
        """
        self.heap = []
        self.heap_size = 0

    def left(self, i):
        """
        Returns the index of the left child node from parent node index.

        Parameters
        ----------
        i: int
            Index of parent node

        Returns
        ----------
        int
            Index of the left child node
        """
        return 2 * i + 1  #located after all nodes on previous levels + node itself

```

```
    def right(self, i):
        """
        Returns the index of the right child node from parent node index.

        Parameters
        ----------
        i: int
            Index of parent node

        Returns
        ----------
        int
            Index of the right child node
        """
        return 2 * i + 2  #located immediatly after left child

    def parent(self, i):
        """
        Returns the index of the parent node from child node index.

        Parameters
        ----------
        i: int
            Index of child node

        Returns
        ----------
        int
            Index of the parent node
        """
        return (i - 1) // 2  #moving up one level; -1 to adjust for 0 indexing

    def maxk(self):
        """
        Returns the highest key in the priority queue.

        Parameters
        ----------
        None

        Returns
        ----------
        int
            The highest key in the priority queue
        """
        if self.heap_size == 0:
            raise ValueError("Heap is empty.")   #error if heap is empty
        return self.heap[0]   #returns root element, which is the max value in maxheap

```

```
    def heappush(self, key):
        """
        Inserts a key into the priority queue.

        Parameters
        ----------
        key: int
            The key value to be inserted

        Returns
        ----------
        None
        """
        self.heap.append(-float("inf"))  #temporary value to maintain heap structure
        self.increase_key(self.heap_size, key)  #set the correct value and adjust position
        self.heap_size += 1

    def increase_key(self, i, key):
        """
        Modifies the value of a key with a higher value to maintain the max-heap property.

        Parameters
        ----------
        i: int
            The index of the key to be modified
        key: int
            The new key value

        Returns
        ----------
        None
        """
        if key < self.heap[i]:
            raise ValueError('New key is smaller than the current key.')

        self.heap[i] = key  #set new value
        #while the current index is not the root and the parent's key is less than current key
        while i > 0 and self.heap[self.parent(i)] < self.heap[i]:
            j = self.parent(i)  #find parent index of current node
            self.heap[i], self.heap[j] = self.heap[j], self.heap[i]  #swap if parent key is smaller
            i = j  #update i to parent's index

```

```python
    def heapify(self, i):
        """
        Maintains the max-heap property for a subtree rooted at index i.

        Parameters
        ----------
        i: int
            The index of the root node of the subtree to be heapified

        Returns
        ----------
        None
        """
        l = self.left(i)
        r = self.right(i)
        largest = i

        #find the largest of root, left child, and right child.
        if l < self.heap_size and self.heap[l] > self.heap[i]:   #check left child exists and is larger than current node
            largest = l   #if true, largest is left child
        if r < self.heap_size and self.heap[r] > self.heap[largest]:
            largest = r   #if true, largest is right child
        #if the largest isn't the current node, swap and continue heapifying.
        if largest != i:
            self.heap[i], self.heap[largest] = self.heap[largest], self.heap[i]
            self.heapify(largest)  #recursively heapify subtree rooted at largest index

    def heappop(self):
        """
        Returns and removes the largest key in the max priority queue.

        Parameters
        ----------
        None

        Returns
        ----------
        int
            The max value in the heap that is extracted
        """
        if self.heap_size < 1:  #error for empty heap
            raise ValueError('Heap underflow: There are no keys in the priority queue.')
        maxk = self.heap[0]  #max is root

        #move the last element to the root and reduce the heap size
        self.heap[0] = self.heap[-1]
        self.heap.pop()  #remove last element (now duplicate with root)
        self.heap_size -= 1  #decrease heap size

        #heapify from the root to maintain max-heap property.
        self.heapify(0)
        return maxk
```

Appendix III a: Testing

Explanation of the Test Cases

1. First test case verifies basic functionality: inserting elements and maintaining max-heap property during extraction.

2. Second test case checks that duplicate values are handled correctly and that max-heap property is preserved even with duplicates.
3. Third test case tests error handling by ensuring that a ValueError is raised when trying to pop from an empty heap.

```
#Testing: basic insertion and max extraction
#create an instance of MaxHeapq
heap = MaxHeapq()

#insert some elements
heap.heappush(10)
heap.heappush(20)
heap.heappush(5)
heap.heappush(15)

#test max extraction
assert heap.heappop() == 20, "Test Case 1 Failed: Expected max to be 20"
assert heap.heappop() == 15, "Test Case 1 Failed: Expected max to be 15"

print("Test Cases Passed!")

```

```
Test Cases Passed!
```

```
#Testing: insert duplicates and maintain heap property
#create another instance of MaxHeapq
heap = MaxHeapq()

#insert duplicate values
heap.heappush(30)
heap.heappush(30)

#test max extraction with duplicates
assert heap.heappop() == 30, "Test Case 2 Failed: Expected max to be 30"
assert heap.heappop() == 30, "Test Case 2 Failed: Expected max to be 30"

print("Test Cases Passed!")

```

```
Test Cases Passed!
```

```python
#Testing: handling empty heap

#extract all elements
heap.heappop()
heap.heappop()

#attempting to pop from an empty heap should raise an error
heap.heappop()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[60], line 4
      1 #Testing: handling empty heap
      2
      3 #extract all elements
----> 4 heap.heappop()
      5 heap.heappop()
      7 #attempting to pop from an empty heap should raise an error

Cell In[46], line 169, in MaxHeapq.heappop(self)
    156 """
    157 Returns and removes the largest key in the max priority queue.
    158
   (...)
    166     The max value in the heap that is extracted
    167 """
    168 if self.heap_size < 1:  #error for empty heap
--> 169     raise ValueError('Heap underflow: There are no keys in the priority queue.')
    170 maxk = self.heap[0]  #max is root
    172 #move the last element to the root and reduce the heap size

ValueError: Heap underflow: There are no keys in the priority queue.
```

Appendix IV: TaskScheduler

The following implementations were adapted from Sousa, M. (2024, October 23). CS110 Session 13 - [7.2] Heaps and priority queues

```python
class Task:
    """
    A class representing a task in the scheduler.

    Attributes
    ----------
    id : int
        Task id (a reference number)
    description : str
        Task short description
    duration : int
        Task duration in minutes
    dependencies : list
        List of task ids that need to precede this task
    status : str
        Current status of the task
    fixed_time : int or None
        The specific time the task should start, in minutes from the start of the day (or None if no fixed time)
    task_deadline : int
        The deadline by which the task should be completed
    importance : str
        The importance level of the task, can be "critical", "moderate", or "low"
    """

    #initializes an instance of Task
    def __init__(self, id, description, duration, dependencies, status="N",
                 fixed_time=None, task_deadline=None, importance="N"):
        self.id = id
        self.description = description
        self.duration = duration
        self.dependencies = dependencies
        self.status = status
        self.fixed_time = fixed_time  #specific time (in minutes) the task should start
        self.task_deadline = task_deadline  #deadline for task completion
        self.importance = importance  #importance level of the task

    def __lt__(self, other):
        #compare tasks based on duration for sorting
        return self.duration < other.duration
```

```python
import random

class TaskScheduler:
    """
    A daily task scheduler using custom max heap queue
    """
    NOT_STARTED = 'N'
    IN_PRIORITY_QUEUE = 'I'
    COMPLETED = 'C'

    def __init__(self, tasks):
        self.tasks = tasks
        self.priority_queue = MaxHeapq()  #using custom max-heap
        self.task_map = {}  #dictionary to map priority values to tasks

    def print_self(self):
        """
        Prints the details of all tasks added to the scheduler
        """
        print("Tasks added to the scheduler:")
        print("-------------------------------------")
        for t in self.tasks:
            print(f"➥ '{t.description}', duration = {t.duration} mins, fixed_time={t.fixed_time}, deadline={t.task_deadline}, importanc
            if len(t.dependencies) > 0:
                print(f"\t ⚠ This task depends on others!")

    def remove_dependency(self, completed_task_id):
        """
        Removes the dependency from all tasks once a specific task is completed.

        Parameters
        ----------
        completed_task_id : int
            The ID of the task that has been completed
        """
        for task in self.tasks:   #iterate over all tasks
            if completed_task_id in task.dependencies:
                task.dependencies.remove(completed_task_id)  #remove completed task from the dependencies lists

```

```python
    def get_tasks_ready(self, current_time):
        """
        Check and add tasks that are ready to the priority queue

        Parameters
        ----------
        current_time : int
            The current time in minutes from the start of the day
        """
        for task in self.tasks:
            if task.status == self.NOT_STARTED and not task.dependencies:   #check tasks that have no dependencies or all dependencies a
                    if task.fixed_time is None or task.fixed_time <= current_time:  #a task with fixed time will only be added if that t
                        if task.status != self.IN_PRIORITY_QUEUE:   #prevents adding tasks multiple times (duplicates)
                            priority = self.calculate_priority(task, current_time)  #calculate priority value for task
                            self.priority_queue.heappush(priority)  #add to priority queue
                            self.task_map[priority] = task
                            task.status = self.IN_PRIORITY_QUEUE  #update task status

```

```python
    def calculate_priority(self, task, current_time):
        """
        Calculates priority based on task attributes.

        Parameters
        ----------
        task : Task
            The task for which to calculate priority
        current_time : int
            The current time in minutes from the start of the day

        Returns
        ----------
        int
            The priority score for the task
        """
        #calculate urgency level based on time remaining until the deadline and how many times that task could be completed until then
        if task.task_deadline is not None:
            urgency = (current_time - task.task_deadline) / task.duration
            urgency_score = 50 if urgency <= 2 else (25 if urgency <= 6 else 0)
        else:
            urgency_score = 0  #no deadline means no urgency

        #calculate importance score based on task importance
        importance_score = {"critical": 40, "moderate": 20, "low": 5}.get(task.importance, 10)

        #duration penalty (longer tasks are deprioritized)
        duration_penalty = task.duration / 2

        #dependency penalty (tasks with more dependencies are deprioritized)
        dependency_penalty = len(task.dependencies) * 10

        #final priority calculation
        priority = importance_score + urgency_score - duration_penalty - dependency_penalty

        #add a small random noise to ensure uniqueness
        noise = random.uniform(0, 0.01)

        return round(priority + noise, 4)

```

```python
    def check_unscheduled_tasks(self):
        """
        Checks if there are any tasks that haven't yet been scheduled

        Returns
        ----------
        bool
            True if there are unscheduled tasks, otherwise False
        """
        for task in self.tasks:
            if task.status == self.NOT_STARTED:
                return True
        return False

    def format_time(self, time):
        """
        Formats the time in minutes into a readable format (e.g. 9h30)

        Parameters
        ----------
        time : int
            The time in minutes to format

        Returns
        ---------
        str
            The formatted time as a string
        """
        return f"{time//60}h{time%60:02d}"  #finds hours dividing by 60, and the minutes are the remainder

```

```python
def run_task_scheduler(self, starting_time):
    """
    Runs the task scheduler starting from given time

    Parameters
    ----------
    starting_time : int
        The starting time in minutes from the start of the day
    """
    current_time = starting_time   #set current time to starting time
    print("Running the scheduler:\n")
    utility_value = 0

    while self.check_unscheduled_tasks() or self.priority_queue.heap_size > 0:   #while there are tasks that haven't been started or 

        #find tasks that are ready to be scheduled
        self.get_tasks_ready(current_time)

        #find the next fixed-time task that hasn't started yet and is scheduled soon
        next_fixed_time_task = None
        for task in self.tasks:
            if task.fixed_time and task.status == self.NOT_STARTED and task.fixed_time > current_time:
                if next_fixed_time_task is None or task.fixed_time < next_fixed_time_task.fixed_time: #ensure this is the earliest fi
                    next_fixed_time_task = task
        #calculate time until the next fixed-time task, if any
        time_until_fixed = next_fixed_time_task.fixed_time - current_time if next_fixed_time_task else float("inf")

        if self.priority_queue.heap_size > 0:
            priority = self.priority_queue.heappop()  #get the highest priority task from the queue
            task = self.task_map.pop(priority)

            if task.duration > time_until_fixed and next_fixed_time_task: #check if the task an be completed before the next fixed-ti
                #if the task can't fit, reinsert it and wait until the next fixed-time task
                self.priority_queue.heappush(priority)  #reinsert the task into the heap
                self.task_map[priority] = task
                current_time = next_fixed_time_task.fixed_time
                continue

            #execute the task
            print(f"🕰 t={self.format_time(current_time)}")
            print(f"\tstarted '{task.description}' for {task.duration} mins, with priority {priority}")
            utility_value += priority
            current_time += task.duration
            print(f"\t✅ t={self.format_time(current_time)}, task completed!")
            task.status = self.COMPLETED  #update status to COMPLETED
            self.remove_dependency(task.id)
            self.get_tasks_ready(current_time)  #re-evaluate tasks after a dependency has been completed

        else:
            #if no tasks are ready, increment time by 5 minutes and try again
            current_time += 5  #increment time by 5 minutes

    total_time = current_time - starting_time
    print(f"\n🏁 Completed all planned tasks in {total_time//60}h{total_time%60:02d}min, and a total utility value of {round(utility_
```

```python
my_tasks = [
    Task(0, "Wake up and get out of bed", 15, [], importance="critical", task_deadline=570),
    Task(1, "Go outside", 20, [0], importance="moderate", task_deadline=900),
    Task(2, "Grab coffee", 20, [0, 1], importance="moderate", task_deadline=940),
    Task(3, "Take CS110 class", 90, [0], fixed_time=10*60, importance="critical", task_deadline=690),
    Task(4, "After Class Break", 30, [0, 3], fixed_time=690, importance="low", task_deadline=720),
    Task(5, "Ride YouBike", 15, [0, 3], importance="low"),
    Task(6, "Grab lunch at a local restaurant", 40, [0, 5], importance="moderate", task_deadline=810),
    Task(7, "Get Boba", 10, [0, 5, 6], importance="low", task_deadline=17*60),
    Task(8, "Complete CS111 PCW", 90, [0,6], importance="critical", task_deadline=23*60)
]

simple_scheduler = TaskScheduler(my_tasks)
simple_scheduler.run_task_scheduler(starting_time=8*60+30)
```

```
Running the scheduler:

🕰️ t=8h30
        started 'Wake up and get out of bed' for 15 mins, with priority 82.5004
        ✅ t=8h45, task completed!
🕰️ t=8h45
        started 'Go outside' for 20 mins, with priority 60.0021
        ✅ t=9h05, task completed!
🕰️ t=9h05
        started 'Grab coffee' for 20 mins, with priority 60.0023
        ✅ t=9h25, task completed!
🕰️ t=10h00
        started 'Take CS110 class' for 90 mins, with priority 45.0025
        ✅ t=11h30, task completed!
🕰️ t=11h30
        started 'After Class Break' for 30 mins, with priority 40.0049
        ✅ t=12h00, task completed!
🕰️ t=12h00
        started 'Ride YouBike' for 15 mins, with priority -2.4935
        ✅ t=12h15, task completed!
🕰️ t=12h15
        started 'Grab lunch at a local restaurant' for 40 mins, with priority 50.0061
        ✅ t=12h55, task completed!
🕰️ t=12h55
        started 'Get Boba' for 10 mins, with priority 50.0024
        ✅ t=13h05, task completed!
🕰️ t=13h05
        started 'Complete CS111 PCW' for 90 mins, with priority 45.0073
        ✅ t=14h35, task completed!

🏁 Completed all planned tasks in 6h05min, and a total utility value of 430.0345!
```

Appendix IV a: Examples Correct Prioritization

```python
#Example for pririotizing based on priority value
task_1 = Task(0, "Task 1 - High Priority", 10, [], importance="critical", task_deadline=600)  #high priority task
task_2 = Task(1, "Task 2 - Moderate Priority", 20, [], importance="moderate", task_deadline=900)  #moderate priority task
task_3 = Task(2, "Task 3 - Low Priority", 30, [], importance="low", task_deadline=1200)  #low priority task

scheduler = TaskScheduler([task_1, task_2, task_3])
scheduler.run_task_scheduler(starting_time=8 * 60) #starting at 8am
```

```
Running the scheduler:

🕰️ t=8h00
        started 'Task 1 - High Priority' for 10 mins, with priority 85.0095
        ✅ t=8h10, task completed!
🕰️ t=8h10
        started 'Task 2 - Moderate Priority' for 20 mins, with priority 60.0003
        ✅ t=8h30, task completed!
🕰️ t=8h30
        started 'Task 3 - Low Priority' for 30 mins, with priority 40.0026
        ✅ t=9h00, task completed!

🏁 Completed all planned tasks in 1h00min, and a total utility value of 185.0124!
```

```python
#Example changing order of input
task_4 = Task(3, "Task 1 - High Priority", 10, [], importance="critical", task_deadline=600)
task_5 = Task(4, "Task 2 - Moderate Priority", 20, [], importance="moderate", task_deadline=900)
task_6 = Task(5, "Task 3 - Low Priority", 30, [], importance="low", task_deadline=1200)

#scheduler with the same tasks in different order
scheduler_reordered = TaskScheduler([task_6, task_4, task_5])
scheduler_reordered.run_task_scheduler(starting_time=8 * 60)

#assert statements to confirm that the output order remains the same
assert scheduler.tasks[0].description == scheduler_reordered.tasks[1].description, "First task should be the high priority task"
assert scheduler.tasks[1].description == scheduler_reordered.tasks[2].description, "Second task should be the moderate priority task"
assert scheduler.tasks[2].description == scheduler_reordered.tasks[0].description, "Third task should be the low priority task"
```

```
Running the scheduler:

🕰️ t=8h00
        started 'Task 1 - High Priority' for 10 mins, with priority 85.0096
        ✅ t=8h10, task completed!
🕰️ t=8h10
        started 'Task 2 - Moderate Priority' for 20 mins, with priority 60.006
        ✅ t=8h30, task completed!
🕰️ t=8h30
        started 'Task 3 - Low Priority' for 30 mins, with priority 40.0078
        ✅ t=9h00, task completed!

🏁 Completed all planned tasks in 1h00min, and a total utility value of 185.0234!
```

Appendix IV b: Example Not Adhering to Deadlines

```python
#ADD TASK
#direspecting deadlines

my_tasks_1 = [
    Task(0, "Wake up and get out of bed", 15, [], importance="critical", task_deadline=570),
    Task(1, "Go outside", 20, [0], importance="moderate", task_deadline=900),
    Task(2, "Grab coffee", 20, [0, 1], importance="moderate", task_deadline=940),
    Task(3, "Take CS110 class", 90, [0], fixed_time=10*60, importance="critical", task_deadline=690),
    Task(4, "After Class Break", 30, [0, 3], fixed_time=690, importance="low", task_deadline=720),
    Task(5, "Ride YouBike", 15, [0, 3], importance="low"),
    Task(6, "Grab lunch at a local restaurant", 40, [0, 5], importance="moderate", task_deadline=810),
    Task(7, "Get Boba", 10, [0, 5, 6], importance="low", task_deadline=17*60),
    Task(8, "Complete CS111 PCW", 90, [0,6], importance="critical", task_deadline=23*60),
    Task(9, "Swimming", 90, [0], importance="moderate", task_deadline=17*60)
]

simple_scheduler = TaskScheduler(my_tasks_1)
simple_scheduler.run_task_scheduler(starting_time=8*60)
```

```
Running the scheduler:

🕰️ t=8h00
        started 'Wake up and get out of bed' for 15 mins, with priority 82.5091
        ✅ t=8h15, task completed!
🕰️ t=8h15
        started 'Go outside' for 20 mins, with priority 60.0018
        ✅ t=8h35, task completed!
🕰️ t=8h35
        started 'Grab coffee' for 20 mins, with priority 60.0091
        ✅ t=8h55, task completed!
🕰️ t=10h00
        started 'Take CS110 class' for 90 mins, with priority 45.0056
        ✅ t=11h30, task completed!
🕰️ t=11h30
        started 'After Class Break' for 30 mins, with priority 40.0082
        ✅ t=12h00, task completed!
🕰️ t=12h00
        started 'Swimming' for 90 mins, with priority 25.0091
        ✅ t=13h30, task completed!
🕰️ t=13h30
        started 'Ride YouBike' for 15 mins, with priority -2.4982
        ✅ t=13h45, task completed!
🕰️ t=13h45
        started 'Grab lunch at a local restaurant' for 40 mins, with priority 50.0033
        ✅ t=14h25, task completed!
🕰️ t=14h25
        started 'Get Boba' for 10 mins, with priority 50.0053
        ✅ t=14h35, task completed!
🕰️ t=14h35
        started 'Complete CS111 PCW' for 90 mins, with priority 45.0095
        ✅ t=16h05, task completed!

🏁 Completed all planned tasks in 8h05min, and a total utility value of 455.0628!
```

Appendix V: Experimental Analysis

```python
import time
import matplotlib.pyplot as plt
import numpy as np
import math

#sample Task class (simplified for testing purposes)
class Task:
    def __init__(self, task_id, description, duration, dependencies=None, fixed_time=None, importance="low", task_deadline=None):
        self.id = task_id
        self.description = description
        self.duration = duration
        self.dependencies = dependencies or []
        self.fixed_time = fixed_time
        self.importance = importance
        self.task_deadline = task_deadline
        self.status = TaskScheduler.NOT_STARTED

#function to generate random tasks for testing
def generate_tasks(num_tasks):
    tasks = []
    for i in range(num_tasks):
        duration = random.randint(5, 60)  #random duration between 5 and 60 mins
        dependencies = random.sample(range(max(0, i - 5), i), k=random.randint(0, min(3, i)))  #random dependencies within recent tasks
        fixed_time = random.choice([None, random.randint(8*60, 23*60)])  #random fixed time or None
        importance = random.choice(["low", "moderate", "critical"])
        task_deadline = random.choice([None, random.randint(9*60, 23*60)])  #random deadline or None
        tasks.append(Task(i, f"Task {i}", duration, dependencies, fixed_time, importance, task_deadline))
    return tasks

#running experiments
task_counts = [10, 20, 40, 80, 120, 200, 350]
times = []
theoretical_times = [] #check if it matches with theoretical time complexity

for num_tasks in task_counts:
    tasks = generate_tasks(num_tasks)
    scheduler = TaskScheduler(tasks)
    start_time = time.time()
    scheduler.run_task_scheduler(starting_time=8 * 60)  # start at 8:00 AM
    end_time = time.time()
    times.append(end_time - start_time)
    theoretical_time = num_tasks ** 2 * math.log(num_tasks + 1)  #+1 to avoid log(0)
    theoretical_times.append(theoretical_time)
    print(f"Completed scheduling for {num_tasks} tasks in {end_time - start_time:.4f} seconds.")

max_time = max(times) #uses actual max runtime as scaling reference
max_theoretical_time = max(theoretical_times) #max in theoretical times
normalized_theoretical_times = [t * (max_time / max_theoretical_time) for t in theoretical_times] #scales theoretical times proportional

#plotting results
plt.plot(task_counts, times, marker='o')
plt.plot(task_counts, normalized_theoretical_times, marker='x', linestyle='--', label=r"$n^2 \log(n)$")
plt.xlabel("Number of Tasks")
plt.ylabel("Time (seconds)")
plt.title("Scheduler Performance with Increasing Number of Tasks")
plt.grid(True)
plt.legend()
plt.show()
```

🏁 Completed all planned tasks in 204h00min, and a total utility value of 3899.2358!
Completed scheduling for 350 tasks in 0.0316 seconds.